

IMPLEMENTATION_ROADMAP

HelixCode Notification Integration - Implementation Roadmap

Version: 1.0 **Date:** 2025-11-04 **Status:** Ready for Execution **Estimated Total Time:** 11 weeks

Table of Contents

- 1. [Current Status](#)
 - 2. [Phase 1: Mock Servers & Testing Infrastructure](#) (Week 1-2)
 - 3. [Phase 2: Event-Driven Hook System](#) (Week 3-4)
 - 4. [Phase 3: Retry Logic & Reliability](#) (Week 5-6)
 - 5. [Phase 4: Generic Webhooks & Microsoft Teams](#) (Week 7)
 - 6. [Phase 5: Advanced Integrations](#) (Week 8-9)
 - 7. [Phase 6: Documentation & Website Polish](#) (Week 10)
 - 8. [Phase 7: Performance & Scale Testing](#) (Week 11)
 - 9. [Quick Reference: Daily Tasks](#)
-

Current Status

Completed (Phase 0)

- Email recipient extraction bug fixed
- Telegram integration implemented
- 100% test coverage for all channels (unit tests)
- Setup guides for Slack, Telegram, Email
- Website updated with integrations section
- Configuration system enhanced

Remaining Work

- Mock server infrastructure for integration tests
- Event-driven hook triggering system
- Retry logic and rate limiting
- Additional integrations (Generic Webhooks, MS Teams, etc.)
- Performance testing and optimization

Phase 1: Mock Servers & Testing Infrastructure

Duration: Week 1-2 (10 days) **Priority:** HIGH **Dependencies:** None **Goal:** Complete integration testing infrastructure

Day 1-2: Mock Server Infrastructure

Task 1.1: Create Mock Server Utility Package

File: helix_code/internal/notification/testutil/mock_servers.go

Implementation:

```
package testutil

import (
    "encoding/json"
    "io"
    "net/http"
    "net/http/httptest"
    "sync"
)

// MockSlackServer simulates Slack webhook endpoint
type MockSlackServer struct {
    *httptest.Server
    Requests []SlackRequest
    mutex     sync.Mutex
}

type SlackRequest struct {
    Channel   string `json:"channel"`
    Username  string `json:"username"`
    Text      string `json:"text"`
    IconEmoji string `json:"icon_emoji"`
}

func NewMockSlackServer() *MockSlackServer {
    mock := &MockSlackServer{
        Requests: make([]SlackRequest, 0),
    }
}
```

```

mock.Server = httptest.NewServer(http.HandlerFunc(func(w
    http.ResponseWriter, r *http.Request) {
        mock.mutex.Lock()
        defer mock.mutex.Unlock()

        // Verify it's POST
        if r.Method != http.MethodPost {
            w.WriteHeader(http.StatusMethodNotAllowed)
            return
        }

        // Parse request
        var req SlackRequest
        body, _ := io.ReadAll(r.Body)
        if err := json.Unmarshal(body, &req); err != nil {
            w.WriteHeader(http.StatusBadRequest)
            return
        }

        mock.Requests = append(mock.Requests, req)
        w.WriteHeader(http.StatusOK)
    }))

return mock
}

func (m *MockSlackServer) GetRequests() []SlackRequest {
    m.mutex.Lock()
    defer m.mutex.Unlock()
    return append([]SlackRequest{}, m.Requests...)
}

func (m *MockSlackServer) Reset() {
    m.mutex.Lock()
    defer m.mutex.Unlock()
    m.Requests = make([]SlackRequest, 0)
}

func (m *MockSlackServer) GetRequestCount() int {
    m.mutex.Lock()
    defer m.mutex.Unlock()
    return len(m.Requests)
}

```

```

// MockTelegramServer simulates Telegram Bot API
type MockTelegramServer struct {
    *httptest.Server
    Requests []TelegramRequest
    mutex     sync.Mutex
}

type TelegramRequest struct {
    ChatID    string `json:"chat_id"`
    Text      string `json:"text"`
    ParseMode string `json:"parse_mode"`
}

func NewMockTelegramServer() *MockTelegramServer {
    mock := &MockTelegramServer{
        Requests: make([]TelegramRequest, 0),
    }

    mock.Server = httptest.NewServer(http.HandlerFunc(func(w
        http.ResponseWriter, r *http.Request) {
            mock.mutex.Lock()
            defer mock.mutex.Unlock()

            // Verify it's POST to /sendMessage
            if r.Method != http.MethodPost {
                w.WriteHeader(http.StatusMethodNotAllowed)
                return
            }

            // Parse request
            var req TelegramRequest
            body, _ := io.ReadAll(r.Body)
            if err := json.Unmarshal(body, &req); err != nil {
                w.WriteHeader(http.StatusBadRequest)
                return
            }

            mock.Requests = append(mock.Requests, req)

            // Return successful Telegram API response
            response := map[string]interface{}{
                "ok": true,
                "result": map[string]interface{}{
                    "message_id": len(mock.Requests),
                },
            }
        })
    })
    return mock
}

```

```

        "chat": map[string]interface{}{
            "id": req.ChatID,
        },
        "text": req.Text,
    },
}

w.Header().Set("Content-Type", "application/json")
json.NewEncoder(w).Encode(response)
}))

return mock
}

func (m *MockTelegramServer) GetRequests() []TelegramRequest {
    m.mutex.Lock()
    defer m.mutex.Unlock()
    return append([]TelegramRequest{}, m.Requests...)
}

func (m *MockTelegramServer) Reset() {
    m.mutex.Lock()
    defer m.mutex.Unlock()
    m.Requests = make([]TelegramRequest, 0)
}

// MockDiscordServer simulates Discord webhook endpoint
type MockDiscordServer struct {
    *httptest.Server
    Requests []DiscordRequest
    mutex     sync.Mutex
}

type DiscordRequest struct {
    Content string `json:"content"`
}

func NewMockDiscordServer() *MockDiscordServer {
    mock := &MockDiscordServer{
        Requests: make([]DiscordRequest, 0),
    }

    mock.Server = httptest.NewServer(http.HandlerFunc(func(w
        http.ResponseWriter, r *http.Request) {

```

```

mock.mutex.Lock()
defer mock.mutex.Unlock()

if r.Method != http.MethodPost {
    w.WriteHeader(http.StatusMethodNotAllowed)
    return
}

var req DiscordRequest
body, _ := io.ReadAll(r.Body)
if err := json.Unmarshal(body, &req); err != nil {
    w.WriteHeader(http.StatusBadRequest)
    return
}

mock.Requests = append(mock.Requests, req)
w.WriteHeader(http.StatusNoContent)
}))

return mock
}

func (m *MockDiscordServer) GetRequests() []DiscordRequest {
    m.mutex.Lock()
    defer m.mutex.Unlock()
    return append([]DiscordRequest{}, m.Requests...)
}

func (m *MockDiscordServer) Reset() {
    m.mutex.Lock()
    defer m.mutex.Unlock()
    m.Requests = make([]DiscordRequest, 0)
}

```

Testing:

```
go test ./internal/notification/testutil/... -v
```

Acceptance Criteria: - Mock servers correctly simulate Slack, Telegram, Discord APIs - Requests are captured and accessible - Thread-safe operations - Reset functionality works - All tests pass

Task 1.2: Create Mock SMTP Server

File: helix_code/internal/notification/testutil/mock_smtp.go

Implementation Options:

Option A: Use MailHog (Recommended)

Start MailHog for testing

```
docker run -d -p 1025:1025 -p 8025:8025 mailhog/mailhog
```

Option B: Implement Simple Mock

```
package testutil
```

```
import (  
    "net"  
    "sync"  
)
```

```
type MockSMTPServer struct {  
    listener net.Listener  
    Emails   []EmailMessage  
    mutex    sync.Mutex  
    running  bool  
}
```

```
type EmailMessage struct {  
    From      string  
    To        []string  
    Subject   string  
    Body      string  
    ReceivedAt time.Time  
}
```

```
func NewMockSMTPServer(port int) (*MockSMTPServer, error) {  
    listener, err := net.Listen("tcp", fmt.Sprintf(":%d", port))  
    if err != nil {  
        return nil, err  
    }  
  
    mock := &MockSMTPServer{  
        listener: listener,  
        Emails:   make([]EmailMessage, 0),  
    }  
}
```

```

    go mock.serve()

    return mock, nil
}

func (m *MockSMTPServer) serve() {
    m.running = true
    for m.running {
        conn, err := m.listener.Accept()
        if err != nil {
            continue
        }
        go m.handleConnection(conn)
    }
}

func (m *MockSMTPServer) handleConnection(conn net.Conn) {
    defer conn.Close()

    // Implement basic SMTP protocol
    // This is a simplified version - use MailHog for production
    // testing
    conn.Write([]byte("220 localhost SMTP Mock\r\n"))

    // Handle EHLO, MAIL FROM, RCPT TO, DATA, QUIT
    // Store email in m.Emails
}

func (m *MockSMTPServer) GetEmails() []EmailMessage {
    m.mutex.Lock()
    defer m.mutex.Unlock()
    return append([]EmailMessage{}, m.Emails...)
}

func (m *MockSMTPServer) Close() error {
    m.running = false
    return m.listener.Close()
}

```

Recommendation: Use MailHog for comprehensive SMTP testing, implement simple mock only if needed.

Acceptance Criteria: - SMTP server accepts connections - Emails are captured - Authentication can be tested - TLS/SSL can be tested

Day 3-4: Integration Tests with Mock Servers

Task 1.3: Slack Integration Tests

File: helix_code/test/integration/slack_integration_test.go

```
//go:build integration
```

```
package integration
```

```
import (  
    "context"  
    "testing"
```

```
    "dev.helix.code/internal/notification"  
    "dev.helix.code/internal/notification/testutil"  
    "github.com/stretchr/testify/assert"  
    "github.com/stretchr/testify/require"  
)
```

```
func TestSlackIntegration(t *testing.T) {  
    // Start mock Slack server  
    mockServer := testutil.NewMockSlackServer()  
    defer mockServer.Close()
```

```
    // Create Slack channel with mock server URL  
    channel := notification.NewSlackChannel(  
        mockServer.URL,  
        "#test-channel",  
        "TestBot",  
    )
```

```
    tests := []struct {  
        name           string  
        notification *notification.Notification  
        wantRequests   int  
    }{  
        {  
            name: "info notification",  
            notification: &notification.Notification{  
                Title:    "Test Info",  
                Message: "This is a test",  
                Type:     notification.NotificationTypeInfo,  
            },  
        },  
    }
```

```

        },
        wantRequests: 1,
    },
    {
        name: "error notification",
        notification: &notification.Notification{
            Title:    "Test Error",
            Message:  "An error occurred",
            Type:     notification.NotificationTypeError,
        },
        wantRequests: 1,
    },
}

for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
        mockServer.Reset()

        err := channel.Send(context.Background(),
            tt.notification)
        require.NoError(t, err)

        requests := mockServer.GetRequests()
        assert.Equal(t, tt.wantRequests, len(requests))

        if len(requests) > 0 {
            req := requests[0]
            assert.Equal(t, "#test-channel", req.Channel)
            assert.Equal(t, "TestBot", req.Username)
            assert.Contains(t, req.Text,
                tt.notification.Title)
        }
    })
}

func TestSlackIntegration_ErrorScenarios(t *testing.T) {
    // Create a server that returns errors
    mockServer := testutil.NewMockSlackServerWithErrors([]int{
        500, // First request: server error
        200, // Second request: success
    })
    defer mockServer.Close()

```

```

channel := notification.NewSlackChannel(
    mockServer.URL,
    "#test",
    "bot",
)

notification := &notification.Notification{
    Title:    "Test",
    Message:  "Test",
    Type:     notification.NotificationTypeInfo,
}

// First attempt should fail
err := channel.Send(context.Background(), notification)
assert.Error(t, err)

// Second attempt should succeed
err = channel.Send(context.Background(), notification)
assert.NoError(t, err)
}

func TestSlackIntegration_RateLimit(t *testing.T) {
    // TODO: Implement after adding rate limiting in Phase 3
    t.Skip("Rate limiting not yet implemented")
}

```

Similar files to create: - telegram_integration_test.go -
email_integration_test.go - discord_integration_test.go

Testing:

```
go test ./test/integration/... -v -tags=integration
```

Acceptance Criteria: - All integration tests pass - Mock servers correctly
simulate real APIs - Error scenarios are tested - Payload validation works

Day 5: Discord Channel Tests

Task 1.4: Complete Discord Testing

File: helix_code/internal/notification/discord_test.go

```
package notification
```

```
import (
```

```

"context"
"testing"

"github.com/stretchr/testify/assert"
"github.com/stretchr/testify/require"
)

func TestNewDiscordChannel(t *testing.T) {
    tests := []struct {
        name          string
        webhook        string
        wantEnabled    bool
    }{
        {
            name:          "valid webhook",
            webhook:       "https://discord.com/api/webhooks/123/abc",
            wantEnabled:   true,
        },
        {
            name:          "empty webhook",
            webhook:       "",
            wantEnabled:   false,
        },
    },
}

for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
        channel := NewDiscordChannel(tt.webhook)
        assert.Equal(t, tt.wantEnabled, channel.IsEnabled())
        assert.Equal(t, "discord", channel.GetName())
    })
}

func TestDiscordChannel_Send(t *testing.T) {
    // Add comprehensive tests similar to Slack
}

func TestDiscordChannel_GetConfig(t *testing.T) {
    channel := NewDiscordChannel("https://discord.com/api/webhooks/test")
    config := channel.GetConfig()
}

```

```
    assert.Equal(t, "https://discord.com/api/webhooks/test",
        config["webhook"])
}
```

Acceptance Criteria: - 100% test coverage for Discord channel - All tests pass

Day 6-7: CI/CD Integration

Task 1.5: GitHub Actions Workflow

File: .github/workflows/notification-tests.yml

```
name: Notification Tests

on:
  push:
    branches: [ main, develop ]
    paths:
      - 'helix_code/internal/notification/**'
      - 'helix_code/test/integration/**'
  pull_request:
    branches: [ main, develop ]

jobs:
  unit-tests:
    name: Unit Tests
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3

      - name: Set up Go
        uses: actions/setup-go@v4
        with:
          go-version: '1.21'

      - name: Run Unit Tests
        working-directory: ./HelixCode
        run: |
          go test ./internal/notification/... -v -cover
          -coverprofile=coverage.out

      - name: Upload Coverage
```

```

    uses: codecov/codecov-action@v3
    with:
      files: ./helix_code/coverage.out
      flags: notifications

integration-tests:
  name: Integration Tests
  runs-on: ubuntu-latest

  steps:
    - uses: actions/checkout@v3

    - name: Set up Go
      uses: actions/setup-go@v4
      with:
        go-version: '1.21'

    - name: Start MailHog (SMTP Mock)
      run: |
        docker run -d -p 1025:1025 -p 8025:8025 mailhog/mailhog

    - name: Run Integration Tests
      working-directory: ./HelixCode
      run: |
        go test ./test/integration/... -v -tags=integration

```

Acceptance Criteria: - CI/CD pipeline runs on every push/PR - All tests run automatically - Coverage reports generated - Test failures block merges

Day 8-10: Buffer & Documentation

Task 1.6: Test Documentation

File: helix_code/docs/TESTING.md

```
# HelixCode Notification Testing Guide
```

```
## Running Tests
```

```
### Unit Tests
```

```
```bash
```

```
cd HelixCode
```

```
go test ./internal/notification/... -v
```

## Integration Tests

```
Start MailHog for email testing
```

```
docker run -d -p 1025:1025 -p 8025:8025 mailhog/mailhog
```

```
Run integration tests
```

```
go test ./test/integration/... -v -tags=integration
```

## Coverage Report

```
go test ./internal/notification/... -coverprofile=coverage.out
```

```
go tool cover -html=coverage.out
```

## Writing New Tests

### Unit Test Template

[Include template]

### Integration Test Template

[Include template]

## Mock Servers

### Available Mocks

- MockSlackServer
- MockTelegramServer
- MockDiscordServer
- MailHog (SMTP)

[Include usage examples]

```
Acceptance Criteria:
```

- Testing documentation complete
- Examples for all test types
- Mock server usage documented

```

```

```
Phase 2: Event-Driven Hook System
```

```
Duration: Week 3-4 (10 days)
Priority: HIGH
Dependencies: Phase 1
Goal: Automatic notification triggering on system events
```

```
Day 1-3: Event Bus Architecture
```

```
Task 2.1: Implement Event Bus
```

```
File: `helix_code/internal/event/bus.go`
```

```
`go
```

```
package event
```

```
import (
 "context"
 "fmt"
 "log"
 "sync"
 "time"
)
```

```
// EventType represents the type of system event
type EventType string
```

```
const (
 // Task events
 EventTaskCreated EventType = "task.created"
 EventTaskAssigned EventType = "task.assigned"
 EventTaskStarted EventType = "task.started"
 EventTaskCompleted EventType = "task.completed"
 EventTaskFailed EventType = "task.failed"
 EventTaskPaused EventType = "task.paused"

 // Workflow events
 EventWorkflowStarted EventType = "workflow.started"
 EventWorkflowCompleted EventType = "workflow.completed"
 EventWorkflowFailed EventType = "workflow.failed"
 EventStepCompleted EventType = "step.completed"
 EventStepFailed EventType = "step.failed"

 // Worker events
 EventWorkerConnected EventType = "worker.connected"
 EventWorkerDisconnected EventType = "worker.disconnected"
```



```

EventWorkerHealthDegraded EventType = "worker.health_degraded"
EventWorkerHeartbeatMissed EventType = "worker.heartbeat_missed"

// API events
EventUserRegistered EventType = "user.registered"
EventUserLogin EventType = "user.login"
EventProjectCreated EventType = "project.created"
EventAuthFailure EventType = "auth.failure"
)

// EventSeverity indicates the importance of an event
type EventSeverity string

const (
 SeverityInfo EventSeverity = "info"
 SeverityWarning EventSeverity = "warning"
 SeverityError EventSeverity = "error"
 SeverityCritical EventSeverity = "critical"
)

// Event represents a system event
type Event struct {
 ID string `json:"id"`
 Type EventType `json:"type"`
 Timestamp time.Time `json:"timestamp"`
 Source string `json:"source"`
 Severity EventSeverity `json:"severity"`
 Data map[string]interface{} `json:"data"`
 UserID string `json:"user_id,omitempty"`
 ProjectID string `json:"project_id,omitempty"`
}

// EventHandler is a function that handles events
type EventHandler func(ctx context.Context, event Event) error

// EventBus manages event subscriptions and publishing
type EventBus struct {
 subscribers map[EventType][]EventHandler
 mutex sync.RWMutex
 async bool
}

// NewEventBus creates a new event bus
func NewEventBus(async bool) *EventBus {

```

```

 return &EventBus{
 subscribers: make(map[EventType][]EventHandler),
 async: async,
 }
}

// Subscribe registers a handler for a specific event type
func (bus *EventBus) Subscribe(eventType EventType, handler EventHandler) {
 bus.mutex.Lock()
 defer bus.mutex.Unlock()

 if _, exists := bus.subscribers[eventType]; !exists {
 bus.subscribers[eventType] = make([]EventHandler, 0)
 }

 bus.subscribers[eventType] = append(bus.subscribers[eventType], handler)
 log.Printf("Event handler subscribed to: %s", eventType)
}

// SubscribeMultiple subscribes a handler to multiple event types
func (bus *EventBus) SubscribeMultiple(eventTypes []EventType, handler EventHandler) {
 for _, eventType := range eventTypes {
 bus.Subscribe(eventType, handler)
 }
}

// SubscribePattern subscribes to events matching a pattern
// e.g., "task.*" matches all task events
func (bus *EventBus) SubscribePattern(pattern string, handler EventHandler) {
 // TODO: Implement pattern matching
 // For now, use SubscribeMultiple with explicit types
}

// Publish sends an event to all registered handlers
func (bus *EventBus) Publish(ctx context.Context, event Event) error {
 bus.mutex.RLock()
 handlers, exists := bus.subscribers[event.Type]
 bus.mutex.RUnlock()

 if !exists || len(handlers) == 0 {
 log.Printf("No subscribers for event type: %s", event.Type)
 return nil
 }
}

```

```

log.Printf("Publishing event: %s to %d handlers", event.Type, len(handlers))

if bus.async {
 // Async: fire and forget
 for _, handler := range handlers {
 h := handler // capture for goroutine
 go func() {
 if err := h(ctx, event); err != nil {
 log.Printf("Error handling event %s: %v", event.Type, err)
 }
 }()
 }
 return nil
} else {
 // Sync: wait for all handlers
 var errors []string
 for _, handler := range handlers {
 if err := handler(ctx, event); err != nil {
 errors = append(errors, err.Error())
 log.Printf("Error handling event %s: %v", event.Type, err)
 }
 }

 if len(errors) > 0 {
 return fmt.Errorf("event handling errors: %v", errors)
 }
 return nil
}
}

// GetSubscriberCount returns the number of subscribers for an event type
func (bus *EventBus) GetSubscriberCount(eventType EventType) int {
 bus.mutex.RLock()
 defer bus.mutex.RUnlock()
 return len(bus.subscribers[eventType])
}

// GetTotalSubscribers returns total number of subscriptions
func (bus *EventBus) GetTotalSubscribers() int {
 bus.mutex.RLock()
 defer bus.mutex.RUnlock()

 total := 0
 for _, handlers := range bus.subscribers {

```

```

 total += len(handlers)
 }
 return total
}

```

**Tests:** helix\_code/internal/event/bus\_test.go

```
package event
```

```
import (
 "context"
 "testing"
 "time"

 "github.com/stretchr/testify/assert"
)

```

```
func TestEventBus_Subscribe(t *testing.T) {
 bus := NewEventBus(false)

 called := false
 handler := func(ctx context.Context, event Event) error {
 called = true
 return nil
 }

 bus.Subscribe(EventTaskCompleted, handler)

 event := Event{
 ID: "test-1",
 Type: EventTaskCompleted,
 Timestamp: time.Now(),
 Source: "test",
 Severity: SeverityInfo,
 }

 err := bus.Publish(context.Background(), event)
 assert.NoError(t, err)
 assert.True(t, called)
}

```

```
func TestEventBus_MultipleHandlers(t *testing.T) {
 bus := NewEventBus(false)

 callCount := 0

```

```

handler1 := func(ctx context.Context, event Event) error {
 callCount++
 return nil
}
handler2 := func(ctx context.Context, event Event) error {
 callCount++
 return nil
}

bus.Subscribe(EventTaskFailed, handler1)
bus.Subscribe(EventTaskFailed, handler2)

event := Event{
 Type: EventTaskFailed,
 Severity: SeverityError,
}

bus.Publish(context.Background(), event)
assert.Equal(t, 2, callCount)
}

func TestEventBus_NoSubscribers(t *testing.T) {
 bus := NewEventBus(false)

 event := Event{
 Type: EventTaskCompleted,
 Severity: SeverityInfo,
 }

 err := bus.Publish(context.Background(), event)
 assert.NoError(t, err) // Should not error
}

func TestEventBus_AsyncMode(t *testing.T) {
 bus := NewEventBus(true)

 called := false
 handler := func(ctx context.Context, event Event) error {
 time.Sleep(100 * time.Millisecond)
 called = true
 return nil
 }

 bus.Subscribe(EventTaskCompleted, handler)

```

```

event := Event{
 Type: EventTaskCompleted,
 Severity: SeverityInfo,
}

// Publish should return immediately (async)
err := bus.Publish(context.Background(), event)
assert.NoError(t, err)

// Handler might not have been called yet
time.Sleep(200 * time.Millisecond)
assert.True(t, called)
}

```

**Acceptance Criteria:** - Event bus supports subscribe/publish - Both sync and async modes work - Multiple handlers per event type - Thread-safe operations - All tests pass

---

## Task 2.2: Global Event Bus Instance

**File:** helix\_code/internal/event/instance.go

```

package event

import "sync"

var (
 globalBus *EventBus
 globalBusOnce sync.Once
)

// GetGlobalBus returns the global event bus instance
func GetGlobalBus() *EventBus {
 globalBusOnce.Do(func() {
 globalBus = NewEventBus(true) // Async by default
 })
 return globalBus
}

// SetGlobalBus sets the global event bus (useful for testing)
func SetGlobalBus(bus *EventBus) {
 globalBus = bus
}

```

**Acceptance Criteria:** - Singleton pattern implemented - Thread-safe initialization - Can be overridden for testing

---

## Day 4-5: Connect Notification Engine to Event Bus

### Task 2.3: Notification Event Handler

**File:** helix\_code/internal/notification/event\_handler.go

```
package notification

import (
 "context"
 "log"

 "dev.helix.code/internal/event"
)

// NotificationEventHandler handles events and triggers
// notifications
type NotificationEventHandler struct {
 engine *NotificationEngine
}

// NewNotificationEventHandler creates a new event handler
func NewNotificationEventHandler(engine *NotificationEngine)
 *NotificationEventHandler {
 return &NotificationEventHandler{
 engine: engine,
 }
}

// HandleEvent processes an event and sends notifications based
// on rules
func (h *NotificationEventHandler) HandleEvent(ctx
 context.Context, evt event.Event) error {
 log.Printf("Handling event for notifications: %s", evt.Type)

 // Convert event to notification
 notification := h.eventToNotification(evt)

 // Send notification (rules will be applied by the engine)
 return h.engine.SendNotification(ctx, notification)
}
```

```

// eventToNotification converts an event to a notification
func (h *NotificationEventHandler) eventToNotification(evt
 event.Event) *Notification {
 notification := &Notification{
 Title: h.getEventTitle(evt),
 Message: h.getEventMessage(evt),
 Type: h.getNotificationType(evt),
 Priority: h.getNotificationPriority(evt),
 Metadata: evt.Data,
 }

 return notification
}

// getEventTitle generates a title from the event
func (h *NotificationEventHandler) getEventTitle(evt
 event.Event) string {
 titles := map[event.EventType]string{
 event.EventTaskCompleted: "Task Completed",
 event.EventTaskFailed: "Task Failed",
 event.EventWorkflowCompleted: "Workflow Completed",
 event.EventWorkflowFailed: "Workflow Failed",
 event.EventWorkerDisconnected: "Worker Disconnected",
 event.EventWorkerHealthDegraded: "Worker Health
 Degraded",
 }

 if title, exists := titles[evt.Type]; exists {
 return title
 }

 return string(evt.Type)
}

// getEventMessage generates a message from the event
func (h *NotificationEventHandler) getEventMessage(evt
 event.Event) string {
 // Extract relevant info from event data
 taskID, _ := evt.Data["task_id"].(string)
 workerID, _ := evt.Data["worker_id"].(string)
 errorMsg, _ := evt.Data["error"].(string)

 switch evt.Type {
 case event.EventTaskFailed:

```



```

 if errorMsg != "" {
 return "Task " + taskID + " failed: " + errorMsg
 }
 return "Task " + taskID + " failed"

 case event.EventTaskCompleted:
 duration, _ := evt.Data["duration"].(string)
 if duration != "" {
 return "Task " + taskID + " completed in " + duration
 }
 return "Task " + taskID + " completed successfully"

 case event.EventWorkerDisconnected:
 return "Worker " + workerID + " has disconnected"

 case event.EventWorkerHealthDegraded:
 return "Worker " + workerID + " health has degraded"

 default:
 return "Event: " + string(evt.Type)
 }
}

// getNotificationType maps event severity to notification type
func (h *NotificationEventHandler) getNotificationType(evt
 event.Event) NotificationType {
 switch evt.Severity {
 case event.SeverityCritical:
 return NotificationTypeAlert
 case event.SeverityError:
 return NotificationTypeError
 case event.SeverityWarning:
 return NotificationTypeWarning
 case event.SeverityInfo:
 return NotificationTypeInfo
 default:
 return NotificationTypeInfo
 }
}

// getNotificationPriority maps event severity to notification
 priority
func (h *NotificationEventHandler) getNotificationPriority(evt
 event.Event) NotificationPriority {

```

```

switch evt.Severity {
case event.SeverityCritical:
 return NotificationPriorityUrgent
case event.SeverityError:
 return NotificationPriorityHigh
case event.SeverityWarning:
 return NotificationPriorityMedium
case event.SeverityInfo:
 return NotificationPriorityLow
default:
 return NotificationPriorityMedium
}
}

// RegisterWithEventBus registers this handler with the global
// event bus
func (h *NotificationEventHandler) RegisterWithEventBus(bus
 *event.EventBus) {
 // Subscribe to all relevant events
 eventTypes := []event.EventType{
 event.EventTaskCompleted,
 event.EventTaskFailed,
 event.EventWorkflowCompleted,
 event.EventWorkflowFailed,
 event.EventWorkerDisconnected,
 event.EventWorkerHealthDegraded,
 }

 for _, eventType := range eventTypes {
 bus.Subscribe(eventType, h.HandleEvent)
 }

 log.Printf("Notification event handler registered for %d
 event types", len(eventTypes))
}

```

**Tests:** helix\_code/internal/notification/event\_handler\_test.go

**Acceptance Criteria:** - Events converted to notifications correctly - Severity mapped to notification type/priority - Event metadata preserved - All tests pass

---

## Day 6-7: Integrate with Task Manager

### Task 2.4: Add Event Emission to Task Manager

**File:** helix\_code/internal/task/manager.go (modify existing)

**Changes needed:**

```
package task

import (
 "dev.helix.code/internal/event"
 // ... other imports
)

type TaskManager struct {
 // ... existing fields
 eventBus *event.EventBus
}

func NewTaskManager(/* existing params */, eventBus
 *event.EventBus) *TaskManager {
 return &TaskManager{
 // ... existing initialization
 eventBus: eventBus,
 }
}

// Modify CreateTask to emit event
func (tm *TaskManager) CreateTask(ctx context.Context, task
 *Task) error {
 // ... existing task creation logic

 // Emit event
 if tm.eventBus != nil {
 evt := event.Event{
 ID: uuid.New().String(),
 Type: event.EventTaskCreated,
 Timestamp: time.Now(),
 Source: "task_manager",
 Severity: event.SeverityInfo,
 Data: map[string]interface{}{
 "task_id": task.ID,
 "task_name": task.Name,
 "created_by": task.CreatedBy,
 },
 }
 tm.eventBus.Emit(evt)
 }
}
```

```

 "priority": task.Priority,
 },
 UserID: task.CreatedBy,
 ProjectID: task.ProjectID,
}
tm.eventBus.Publish(ctx, evt)
}

return nil
}

// Modify UpdateTaskStatus to emit events
func (tm *TaskManager) UpdateTaskStatus(ctx context.Context,
 taskID string, status TaskStatus) error {
 // ... existing status update logic

 // Emit appropriate event based on status
 if tm.eventBus != nil {
 var eventType event.EventType
 var severity event.EventSeverity

 switch status {
 case TaskStatusRunning:
 eventType = event.EventTaskStarted
 severity = event.SeverityInfo
 case TaskStatusCompleted:
 eventType = event.EventTaskCompleted
 severity = event.SeverityInfo
 case TaskStatusFailed:
 eventType = event.EventTaskFailed
 severity = event.SeverityError
 case TaskStatusPaused:
 eventType = event.EventTaskPaused
 severity = event.SeverityWarning
 default:
 return nil // No event for other statuses
 }

 evt := event.Event{
 ID: uuid.New().String(),
 Type: eventType,
 Timestamp: time.Now(),
 Source: "task_manager",
 Severity: severity,
 }
 }
}

```

```

 Data: map[string]interface{}{
 "task_id": taskID,
 "status": status,
 "duration": /* calculate duration */,
 "error": /* include error if failed */,
 },
 }
 tm.eventBus.Publish(ctx, evt)
}

return nil
}

```

### Testing:

```

func TestTaskManager_EmitsEvents(t *testing.T) {
 bus := event.NewEventBus(false)
 taskManager := NewTaskManager(/* ... */, bus)

 eventReceived := false
 bus.Subscribe(event.EventTaskCreated, func(ctx
 context.Context, evt event.Event) error {
 eventReceived = true
 assert.Equal(t, event.EventTaskCreated, evt.Type)
 return nil
 })

 task := &Task{
 Name: "Test Task",
 }
 err := taskManager.CreateTask(context.Background(), task)
 require.NoError(t, err)
 assert.True(t, eventReceived)
}

```

**Acceptance Criteria:** - Task creation emits EventTaskCreated - Task status changes emit appropriate events - Event data includes all relevant information - Tests verify event emission

---

## Day 8-9: Integrate with Workflow and Worker

### Task 2.5: Add Event Emission to Workflow Executor

**File:** helix\_code/internal/workflow/executor.go (modify existing)

Similar changes as Task Manager: - Emit EventWorkflowStarted - Emit EventWorkflowCompleted - Emit EventWorkflowFailed - Emit EventStepCompleted - Emit EventStepFailed

## Task 2.6: Add Event Emission to Worker Pool

**File:** helix\_code/internal/worker/pool.go (modify existing)

- Emit EventWorkerConnected
- Emit EventWorkerDisconnected
- Emit EventWorkerHealthDegraded
- Emit EventWorkerHeartbeatMissed

**Acceptance Criteria:** - All components emit relevant events - Event data is complete and accurate - Tests verify event emission

---

## Day 10: End-to-End Testing

### Task 2.7: E2E Event Flow Tests

**File:** helix\_code/test/e2e/event\_notification\_e2e\_test.go

```
//go:build e2e
```

```
package e2e
```

```
import (
 "context"
 "testing"
 "time"

 "dev.helix.code/internal/event"
 "dev.helix.code/internal/notification"
 "dev.helix.code/internal/notification/testutil"
 "dev.helix.code/internal/task"
 "github.com/stretchr/testify/assert"
 "github.com/stretchr/testify/require"
)
```

```
func TestE2E_TaskFailure_SendsNotification(t *testing.T) {
 // Setup
 ctx := context.Background()
 bus := event.NewEventBus(false)
```

```

// Setup notification engine with mock Slack
mockSlack := testutil.NewMockSlackServer()
defer mockSlack.Close()

engine := notification.NewNotificationEngine()
slackChannel := notification.NewSlackChannel(mockSlack.URL,
 "#test", "bot")
engine.RegisterChannel(slackChannel)

// Add rule for task failures
rule := notification.NotificationRule{
 Name: "Task Failures",
 Condition: "type==error",
 Channels: []string{"slack"},
 Priority: notification.NotificationPriorityHigh,
 Enabled: true,
}
engine.AddRule(rule)

// Register notification handler with event bus
handler := notification.NewNotificationEventHandler(engine)
handler.RegisterWithEventBus(bus)

// Create task manager with event bus
taskManager := task.NewTaskManager(/* ... */, bus)

// Execute: Create and fail a task
testTask := &task.Task{
 Name: "Test Task",
 Priority: task.PriorityHigh,
}

err := taskManager.CreateTask(ctx, testTask)
require.NoError(t, err)

// Fail the task
err = taskManager.UpdateTaskStatus(ctx, testTask.ID,
 task.TaskStatusFailed)
require.NoError(t, err)

// Wait a bit for async processing
time.Sleep(100 * time.Millisecond)

// Verify: Slack received notification

```

```

 requests := mockSlack.GetRequests()
 require.Equal(t, 1, len(requests), "Should have received 1
 Slack notification")

 slackMsg := requests[0]
 assert.Contains(t, slackMsg.Text, "Task Failed")
 assert.Equal(t, ":x:", slackMsg.IconEmoji)
}

func TestE2E_WorkflowCompletion_SendsNotification(t *testing.T) {
 // Similar test for workflow completion
}

func TestE2E_WorkerDisconnect_SendsAlert(t *testing.T) {
 // Similar test for worker disconnect
}

```

#### Run E2E tests:

```
go test ./test/e2e/... -v -tags=e2e
```

**Acceptance Criteria:** - End-to-end flow works: event → notification → channel  
 - Rules are applied correctly - Multiple channels receive notifications -  
 All E2E tests pass

---

## Phase 3: Retry Logic & Reliability

**Duration:** Week 5-6 (10 days) **Priority:** HIGH **Dependencies:** Phase 1, 2 **Goal:**  
 Ensure reliable notification delivery

### Day 1-3: Retry Mechanism

#### Task 3.1: Implement Retry Logic

**File:** helix\_code/internal/notification/retry.go

```
package notification
```

```
import (
 "context"
 "fmt"
 "log"
 "time"
)
```



```

// RetryConfig defines retry behavior
type RetryConfig struct {
 MaxAttempts int `yaml:"max_attempts"`
 InitialDelay time.Duration `yaml:"initial_delay"`
 MaxDelay time.Duration `yaml:"max_delay"`
 BackoffMultiplier float64 `yaml:"backoff_multiplier"`
}

// DefaultRetryConfig returns sensible defaults
func DefaultRetryConfig() RetryConfig {
 return RetryConfig{
 MaxAttempts: 3,
 InitialDelay: 1 * time.Second,
 MaxDelay: 30 * time.Second,
 BackoffMultiplier: 2.0,
 }
}

// RetryableChannel wraps a channel with retry logic
type RetryableChannel struct {
 channel NotificationChannel
 config RetryConfig
}

// NewRetryableChannel creates a channel wrapper with retry
func NewRetryableChannel(channel NotificationChannel, config
 RetryConfig) *RetryableChannel {
 return &RetryableChannel{
 channel: channel,
 config: config,
 }
}

// Send attempts to send with exponential backoff retry
func (rc *RetryableChannel) Send(ctx context.Context,
 notification *Notification) error {
 var lastErr error
 delay := rc.config.InitialDelay

 for attempt := 1; attempt <= rc.config.MaxAttempts; attempt++ {
 log.Printf("Sending notification via %s (attempt %d/%d)",
 rc.channel.GetName(), attempt, rc.config.MaxAttempts)
 }
}

```

```

err := rc.channel.Send(ctx, notification)
if err == nil {
 // Success!
 if attempt > 1 {
 log.Printf("Notification sent successfully on
attempt %d", attempt)
 }
 return nil
}

lastErr = err
log.Printf("Failed to send notification via %s (attempt
%d/%d): %v",
 rc.channel.GetName(), attempt,
 rc.config.MaxAttempts, err)

// Don't retry on certain errors (e.g., 400 Bad Request)
if !isRetryableError(err) {
 log.Printf("Non-retryable error, giving up: %v", err)
 return err
}

// Last attempt, don't sleep
if attempt == rc.config.MaxAttempts {
 break
}

// Wait before retry with exponential backoff
select {
case <-ctx.Done():
 return ctx.Err()
case <-time.After(delay):
 // Calculate next delay
 delay = time.Duration(float64(delay) *
rc.config.BackoffMultiplier)
 if delay > rc.config.MaxDelay {
 delay = rc.config.MaxDelay
 }
}
}

return fmt.Errorf("failed after %d attempts: %w",
 rc.config.MaxAttempts, lastErr)
}

```

```

// isRetryableError determines if an error should trigger a retry
func isRetryableError(err error) bool {
 errStr := err.Error()

 // Don't retry client errors (4xx)
 if contains(errStr, "status 400") || contains(errStr,
 "status 401") ||
 contains(errStr, "status 403") || contains(errStr,
 "status 404") {
 return false
 }

 // Retry server errors (5xx), network errors, timeouts
 if contains(errStr, "status 5") || contains(errStr,
 "connection") ||
 contains(errStr, "timeout") || contains(errStr, "EOF") {
 return true
 }

 // Default: retry
 return true
}

func contains(s, substr string) bool {
 return len(s) >= len(substr) &&
 (s == substr || len(s) > len(substr) &&
 (s[:len(substr)] == substr || s[len(s)-len(substr):] ==
 substr ||
 s[len(s)/2-len(substr)/2:len(s)/2+len(substr)/
 2+len(substr)%2] == substr))
}

// Implement NotificationChannel interface
func (rc *RetryableChannel) GetName() string {
 return rc.channel.GetName()
}

func (rc *RetryableChannel) IsEnabled() bool {
 return rc.channel.IsEnabled()
}

func (rc *RetryableChannel) GetConfig() map[string]interface{} {
 config := rc.channel.GetConfig()
 config["retry"] = map[string]interface{}{}
}

```

```

 "max_attempts": rc.config.MaxAttempts,
 "initial_delay": rc.config.InitialDelay.String(),
 "max_delay": rc.config.MaxDelay.String(),
 "backoff_multiplier": rc.config.BackoffMultiplier,
 }
 return config
}

```

**Tests:** helix\_code/internal/notification/retry\_test.go

```
package notification
```

```
import (
 "context"
 "errors"
 "testing"
 "time"

 "github.com/stretchr/testify/assert"
 "github.com/stretchr/testify/require"
)

```

```
type mockChannelWithErrors struct {
 name string
 attempts int
 failUntil int
 lastNotification *Notification
}

```

```
func (m *mockChannelWithErrors) Send(ctx context.Context,
 notification *Notification) error {
 m.attempts++
 m.lastNotification = notification

 if m.attempts <= m.failUntil {
 return errors.New("temporary error")
 }
 return nil
}

```

```
func (m *mockChannelWithErrors) GetName() string { return
 m.name }

```

```
func (m *mockChannelWithErrors) IsEnabled() bool { return
 true }

```

```

func (m *mockChannelWithErrors) GetConfig()
 map[string]interface{} { return nil }

func TestRetryableChannel_SuccessFirstAttempt(t *testing.T) {
 mock := &mockChannelWithErrors{name: "test", failUntil: 0}

 config := RetryConfig{
 MaxAttempts: 3,
 InitialDelay: 10 * time.Millisecond,
 }

 channel := NewRetryableChannel(mock, config)

 notification := &Notification{
 Title: "Test",
 Message: "Test",
 Type: NotificationTypeInfo,
 }

 err := channel.Send(context.Background(), notification)
 require.NoError(t, err)
 assert.Equal(t, 1, mock.attempts)
}

func TestRetryableChannel_SuccessAfterRetries(t *testing.T) {
 mock := &mockChannelWithErrors{name: "test", failUntil: 2}

 config := RetryConfig{
 MaxAttempts: 3,
 InitialDelay: 10 * time.Millisecond,
 MaxDelay: 100 * time.Millisecond,
 BackoffMultiplier: 2.0,
 }

 channel := NewRetryableChannel(mock, config)

 notification := &Notification{
 Title: "Test",
 Message: "Test",
 Type: NotificationTypeInfo,
 }

 start := time.Now()
 err := channel.Send(context.Background(), notification)

```

```

duration := time.Since(start)

require.NoError(t, err)
assert.Equal(t, 3, mock.attempts)

// Should have waited ~30ms (10ms + 20ms)
assert.True(t, duration >= 30*time.Millisecond)
}

func TestRetryableChannel_FailsAfterMaxAttempts(t *testing.T) {
 mock := &mockChannelWithErrors{name: "test", failUntil: 10}

 config := RetryConfig{
 MaxAttempts: 3,
 InitialDelay: 10 * time.Millisecond,
 }

 channel := NewRetryableChannel(mock, config)

 notification := &Notification{
 Title: "Test",
 Message: "Test",
 Type: NotificationTypeInfo,
 }

 err := channel.Send(context.Background(), notification)
 require.Error(t, err)
 assert.Contains(t, err.Error(), "failed after 3 attempts")
 assert.Equal(t, 3, mock.attempts)
}

```

**Acceptance Criteria:** - Retry logic works with exponential backoff - Non-retryable errors don't trigger retries - Maximum attempts enforced - Context cancellation respected - All tests pass

---

## Day 4-5: Rate Limiting

### Task 3.2: Implement Rate Limiter

**File:** helix\_code/internal/notification/ratelimit.go

```
package notification
```

```
import (
```

```

 "context"
 "sync"
 "time"
)

// RateLimit defines rate limiting configuration
type RateLimit struct {
 MaxPerSecond int `yaml:"max_per_second"`
 MaxPerMinute int `yaml:"max_per_minute"`
 BurstSize int `yaml:"burst_size"`
}

// RateLimiter implements token bucket algorithm
type RateLimiter struct {
 tokens int
 maxTokens int
 refillRate time.Duration
 refillAmount int
 lastRefill time.Time
 mutex sync.Mutex
}

// NewRateLimiter creates a rate limiter
func NewRateLimiter(maxPerSecond int) *RateLimiter {
 return &RateLimiter{
 tokens: maxPerSecond,
 maxTokens: maxPerSecond,
 refillRate: 1 * time.Second,
 refillAmount: maxPerSecond,
 lastRefill: time.Now(),
 }
}

// Allow checks if a request can proceed
func (rl *RateLimiter) Allow() bool {
 rl.mutex.Lock()
 defer rl.mutex.Unlock()

 // Refill tokens
 now := time.Now()
 elapsed := now.Sub(rl.lastRefill)
 if elapsed >= rl.refillRate {
 periodsElapsed := int(elapsed / rl.refillRate)
 tokensToAdd := periodsElapsed * rl.refillAmount
 }
}

```

```

 rl.tokens += tokensToAdd
 if rl.tokens > rl.maxTokens {
 rl.tokens = rl.maxTokens
 }
 rl.lastRefill = now
 }

 // Check if we have tokens
 if rl.tokens > 0 {
 rl.tokens--
 return true
 }

 return false
}

// Wait blocks until a token is available or context is cancelled
func (rl *RateLimiter) Wait(ctx context.Context) error {
 for {
 if rl.Allow() {
 return nil
 }

 select {
 case <-ctx.Done():
 return ctx.Err()
 case <-time.After(100 * time.Millisecond):
 // Check again
 }
 }
}

// RateLimitedChannel wraps a channel with rate limiting
type RateLimitedChannel struct {
 channel NotificationChannel
 rateLimiter *RateLimiter
}

// NewRateLimitedChannel creates a rate-limited channel
func NewRateLimitedChannel(channel NotificationChannel,
 maxPerSecond int) *RateLimitedChannel {
 return &RateLimitedChannel{
 channel: channel,
 rateLimiter: NewRateLimiter(maxPerSecond),
 }
}

```



```

 }
}

// Send sends with rate limiting
func (rlc *RateLimitedChannel) Send(ctx context.Context,
 notification *Notification) error {
 // Wait for rate limit
 if err := rlc.rateLimiter.Wait(ctx); err != nil {
 return err
 }

 // Send notification
 return rlc.channel.Send(ctx, notification)
}

// Implement NotificationChannel interface
func (rlc *RateLimitedChannel) GetName() string {
 return rlc.channel.GetName()
}

func (rlc *RateLimitedChannel) IsEnabled() bool {
 return rlc.channel.IsEnabled()
}

func (rlc *RateLimitedChannel) GetConfig()
 map[string]interface{} {
 config := rlc.channel.GetConfig()
 config["rate_limit"] = map[string]interface{}{
 "max_per_second": rlc.rateLimiter.maxTokens,
 }
 return config
}

```

**Tests:** Test rate limiter ensures limits are enforced

**Acceptance Criteria:** - Rate limiting enforced - Token bucket algorithm implemented - Context cancellation supported - Tests verify rate limits

---

## Day 6-7: Notification Queue

### Task 3.3: Redis-backed Notification Queue

**File:** helix\_code/internal/notification/queue.go

```

package notification

import (
 "context"
 "encoding/json"
 "fmt"
 "time"

 "github.com/go-redis/redis/v8"
 "github.com/google/uuid"
)

// NotificationQueue manages a persistent notification queue
type NotificationQueue struct {
 redis *redis.Client
 queueKey string
 retryKey string
 failedKey string
}

// NewNotificationQueue creates a Redis-backed queue
func NewNotificationQueue(redisClient *redis.Client)
 *NotificationQueue {
 return &NotificationQueue{
 redis: redisClient,
 queueKey: "helix:notifications:queue",
 retryKey: "helix:notifications:retry",
 failedKey: "helix:notifications:failed",
 }
}

// QueuedNotification represents a notification in the queue
type QueuedNotification struct {
 ID string `json:"id"`
 Notification *Notification `json:"notification"`
 Channels []string `json:"channels"`
 Attempts int `json:"attempts"`
 MaxAttempts int `json:"max_attempts"`
 QueuedAt time.Time `json:"queued_at"`
 LastAttempt time.Time `json:"last_attempt,omitempty"`
}

// Enqueue adds a notification to the queue

```

```

func (q *NotificationQueue) Enqueue(ctx context.Context,
 notification *Notification, channels []string) error {
 qn := &QueuedNotification{
 ID: uuid.New().String(),
 Notification: notification,
 Channels: channels,
 Attempts: 0,
 MaxAttempts: 3,
 QueuedAt: time.Now(),
 }

 data, err := json.Marshal(qn)
 if err != nil {
 return fmt.Errorf("failed to marshal notification: %w",
 err)
 }

 return q.redis.LPush(ctx, q.queueKey, data).Err()
}

// Dequeue retrieves the next notification from the queue
func (q *NotificationQueue) Dequeue(ctx context.Context)
 (*QueuedNotification, error) {
 data, err := q.redis.BRPop(ctx, 5*time.Second,
 q.queueKey).Result()
 if err == redis.Nil {
 return nil, nil // No items
 }
 if err != nil {
 return nil, err
 }

 var qn QueuedNotification
 if err := json.Unmarshal([]byte(data[1]), &qn); err != nil {
 return nil,
 fmt.Errorf("failed to unmarshal notification: %w", err)
 }

 return &qn, nil
}

// Retry moves a notification to the retry queue
func (q *NotificationQueue) Retry(ctx context.Context, qn
 *QueuedNotification) error {
 qn.Attempts++

```

```

qn.LastAttempt = time.Now()

if qn.Attempts >= qn.MaxAttempts {
 // Move to failed queue
 return q.MoveFailed(ctx, qn)
}

data, err := json.Marshal(qn)
if err != nil {
 return err
}

// Add to retry queue with delay
score := float64(time.Now().Add(1 * time.Minute).Unix())
return q.redis.ZAdd(ctx, q.retryKey, &redis.Z{
 Score: score,
 Member: data,
}).Err()
}

// MoveFailed moves a notification to the failed queue
func (q *NotificationQueue) MoveFailed(ctx context.Context, qn
 *QueuedNotification) error {
 data, err := json.Marshal(qn)
 if err != nil {
 return err
 }

 return q.redis.LPush(ctx, q.failedKey, data).Err()
}

// ProcessRetryQueue processes items in the retry queue
func (q *NotificationQueue) ProcessRetryQueue(ctx
 context.Context) error {
 now := float64(time.Now().Unix())

 // Get items ready for retry
 items, err := q.redis.ZRangeByScore(ctx, q.retryKey,
 &redis.ZRangeBy{
 Min: "0",
 Max: fmt.Sprintf("%f", now),
 }).Result()

 if err != nil {

```

```

 return err
}

for _, item := range items {
 // Move back to main queue
 if err := q.redis.LPush(ctx, q.queueKey, item).Err();
 err != nil {
 return err
 }

 // Remove from retry queue
 if err := q.redis.ZRem(ctx, q.retryKey, item).Err();
 err != nil {
 return err
 }
}

return nil
}

// GetQueueStats returns statistics about the queues
func (q *NotificationQueue) GetQueueStats(ctx context.Context)
 (map[string]int64, error) {
 stats := make(map[string]int64)

 queueLen, err := q.redis.LLen(ctx, q.queueKey).Result()
 if err != nil {
 return nil, err
 }
 stats["queue"] = queueLen

 retryLen, err := q.redis.ZCard(ctx, q.retryKey).Result()
 if err != nil {
 return nil, err
 }
 stats["retry"] = retryLen

 failedLen, err := q.redis.LLen(ctx, q.failedKey).Result()
 if err != nil {
 return nil, err
 }
 stats["failed"] = failedLen
}

```

```
 return stats, nil
}
```

### Worker to process queue:

**File:** helix\_code/internal/notification/queue\_worker.go

```
package notification
```

```
import (
 "context"
 "log"
 "time"
)
```

```
// QueueWorker processes notifications from the queue
```

```
type QueueWorker struct {
 queue *NotificationQueue
 engine *NotificationEngine
 stop chan struct{}
}
```

```
// NewQueueWorker creates a queue worker
```

```
func NewQueueWorker(queue *NotificationQueue, engine
 *NotificationEngine) *QueueWorker {
 return &QueueWorker{
 queue: queue,
 engine: engine,
 stop: make(chan struct{}),
 }
}
```

```
// Start begins processing the queue
```

```
func (w *QueueWorker) Start(ctx context.Context) {
 log.Println("Notification queue worker started")

 // Process retry queue periodically
 go w.processRetryQueue(ctx)

 // Main processing loop
 for {
 select {
 case <-w.stop:
 log.Println("Notification queue worker stopped")
 return
 }
```

```

case <-ctx.Done():
 log.Println("Notification queue worker context
cancelled")
 return
default:
 // Dequeue and process
 qn, err := w.queue.Dequeue(ctx)
 if err != nil {
 log.Printf("Error dequeuing notification: %v",
err)
 time.Sleep(1 * time.Second)
 continue
 }

 if qn == nil {
 // No items, wait a bit
 continue
 }

 // Process notification
 if err := w.engine.SendDirect(ctx, qn.Notification,
qn.Channels); err != nil {
 log.Printf("Failed to send queued notification:
%v", err)
 // Retry
 if err := w.queue.Retry(ctx, qn); err != nil {
 log.Printf("Failed to queue for retry: %v",
err)
 }
 } else {
 log.Printf("Successfully sent queued
notification: %s", qn.ID)
 }
}
}

// Stop stops the worker
func (w *QueueWorker) Stop() {
 close(w.stop)
}

// processRetryQueue periodically moves items from retry to main
queue
func (w *QueueWorker) processRetryQueue(ctx context.Context) {

```

```

ticker := time.NewTicker(1 * time.Minute)
defer ticker.Stop()

for {
 select {
 case <-w.stop:
 return
 case <-ctx.Done():
 return
 case <-ticker.C:
 if err := w.queue.ProcessRetryQueue(ctx); err != nil
 {
 log.Printf("Error processing retry queue: %v",
 err)
 }
 }
}

```

**Acceptance Criteria:** - Notifications queued in Redis - Worker processes queue - Failed notifications moved to DLQ - Retry queue processed - Stats available

---

## Day 8-10: Observability

### Task 3.4: Metrics & Monitoring

**File:** helix\_code/internal/notification/metrics.go

```
package notification
```

```

import (
 "sync"
 "time"
)

// Metrics tracks notification statistics
type Metrics struct {
 TotalSent int64
 TotalFailed int64
 ByChannel map[string]*ChannelMetrics
 mutex sync.RWMutex
}

```



```

type ChannelMetrics struct {
 Sent int64
 Failed int64
 LastSuccess time.Time
 LastFailure time.Time
 AvgLatency time.Duration
 totalLatency time.Duration
 latencyCount int64
}

func NewMetrics() *Metrics {
 return &Metrics{
 ByChannel: make(map[string]*ChannelMetrics),
 }
}

func (m *Metrics) RecordSent(channel string, latency
 time.Duration) {
 m.mutex.Lock()
 defer m.mutex.Unlock()

 m.TotalSent++

 if _, exists := m.ByChannel[channel]; !exists {
 m.ByChannel[channel] = &ChannelMetrics{}
 }

 cm := m.ByChannel[channel]
 cm.Sent++
 cm.LastSuccess = time.Now()
 cm.totalLatency += latency
 cm.latencyCount++
 cm.AvgLatency = cm.totalLatency /
 time.Duration(cm.latencyCount)
}

func (m *Metrics) RecordFailed(channel string) {
 m.mutex.Lock()
 defer m.mutex.Unlock()

 m.TotalFailed++

 if _, exists := m.ByChannel[channel]; !exists {
 m.ByChannel[channel] = &ChannelMetrics{}
 }
}

```

```

 }

 cm := m.ByChannel[channel]
 cm.Failed++
 cm.LastFailure = time.Now()
}

func (m *Metrics) GetStats() map[string]interface{} {
 m.mutex.RLock()
 defer m.mutex.RUnlock()

 stats := map[string]interface{}{
 "total_sent": m.TotalSent,
 "total_failed": m.TotalFailed,
 "by_channel": make(map[string]interface{}),
 }

 for channel, metrics := range m.ByChannel {
 stats["by_channel"].(map[string]interface{})[channel] =
 map[string]interface{}{
 "sent": metrics.Sent,
 "failed": metrics.Failed,
 "last_success": metrics.LastSuccess,
 "last_failure": metrics.LastFailure,
 "avg_latency": metrics.AvgLatency.String(),
 }
 }

 return stats
}

```

#### Add to NotificationEngine:

```

type NotificationEngine struct {
 // ... existing fields
 metrics *Metrics
}

func (e *NotificationEngine) sendToChannels(ctx context.Context,
 notification *Notification) error {
 // ... existing code

 for _, channelName := range notification.Channels {
 channel, exists := e.channels[channelName]
 if !exists || !channel.IsEnabled() {

```

```

 continue
 }

 start := time.Now()
 if err := channel.Send(ctx, notification); err != nil {
 e.metrics.RecordFailed(channelName)
 errors = append(errors, fmt.Sprintf("%s: %v",
 channelName, err))
 } else {
 latency := time.Since(start)
 e.metrics.RecordSent(channelName, latency)
 log.Printf("Notification sent via %s in %v",
 channelName, latency)
 }
}

// ... rest of code
}

```

### API endpoint for metrics:

**File:** helix\_code/internal/api/handlers/notification\_stats.go

```

package handlers

import (
 "net/http"

 "dev.helix.code/internal/notification"
 "github.com/gin-gonic/gin"
)

func GetNotificationStats(engine
 *notification.NotificationEngine) gin.HandlerFunc {
 return func(c *gin.Context) {
 stats := engine.GetMetrics().GetStats()
 c.JSON(http.StatusOK, stats)
 }
}

```

**Acceptance Criteria:** - Metrics tracked for all notifications - Per-channel statistics available - Latency tracking - API endpoint for stats - Grafana dashboard (optional)

---

## Phase 4: Generic Webhooks & Microsoft Teams

**Duration:** Week 7 (5 days) **Priority:** MEDIUM **Dependencies:** Phase 1-3 **Goal:** Add flexible webhook support and MS Teams integration

### Day 1-3: Generic Webhooks

#### Task 4.1: Implement Generic Webhook Channel

**File:** helix\_code/internal/notification/webhook.go

```
package notification

import (
 "bytes"
 "context"
 "encoding/json"
 "fmt"
 "html/template"
 "net/http"
)

// WebhookChannel implements generic webhook notifications
type WebhookChannel struct {
 name string
 enabled bool
 url string
 method string // POST, PUT, PATCH
 headers map[string]string
 payloadTemplate *template.Template
 authType string // none, bearer, basic, apikey
 authToken string
 verifySSL bool
}

// WebhookConfig for creating webhook channel
type WebhookConfig struct {
 Name string
 URL string
 Method string
 Headers map[string]string
 PayloadTemplate string
 AuthType string
 AuthToken string
}
```

```

 VerifySSL bool
}

// NewWebhookChannel creates a generic webhook channel
func NewWebhookChannel(config WebhookConfig) (*WebhookChannel,
 error) {
 // Default to POST
 if config.Method == "" {
 config.Method = "POST"
 }

 // Parse template if provided
 var tpl *template.Template
 var err error
 if config.PayloadTemplate != "" {
 tpl, err =
 template.New("webhook").Parse(config.PayloadTemplate)
 if err != nil {
 return nil, fmt.Errorf("invalid payload template:
 %w", err)
 }
 }

 return &WebhookChannel{
 name: config.Name,
 enabled: config.URL != "",
 url: config.URL,
 method: config.Method,
 headers: config.Headers,
 payloadTemplate: tpl,
 authType: config.AuthType,
 authToken: config.AuthToken,
 verifySSL: config.VerifySSL,
 }, nil
}

// Send sends notification via webhook
func (c *WebhookChannel) Send(ctx context.Context, notification
 *Notification) error {
 if !c.enabled {
 return fmt.Errorf("webhook channel disabled")
 }

 // Build payload
 var payload []byte

```

```

var err error

if c.payloadTemplate != nil {
 // Use custom template
 var buf bytes.Buffer
 if err := c.payloadTemplate.Execute(&buf, notification);
 err != nil {
 return fmt.Errorf("failed to execute template: %w",
 err)
 }
 payload = buf.Bytes()
} else {
 // Default JSON payload
 payload, err = json.Marshal(map[string]interface{}{
 "title": notification.Title,
 "message": notification.Message,
 "type": notification.Type,
 "priority": notification.Priority,
 "metadata": notification.Metadata,
 })
 if err != nil {
 return fmt.Errorf("failed to marshal payload: %w",
 err)
 }
}

// Create request
req, err := http.NewRequestWithContext(ctx, c.method, c.url,
 bytes.NewReader(payload))
if err != nil {
 return fmt.Errorf("failed to create request: %w", err)
}

// Set headers
req.Header.Set("Content-Type", "application/json")
for key, value := range c.headers {
 req.Header.Set(key, value)
}

// Set authentication
switch c.authType {
case "bearer":
 req.Header.Set("Authorization", "Bearer "+c.authToken)
case "basic":
 // Assume token is base64 encoded "user:pass"

```

```

 req.Header.Set("Authorization", "Basic "+c.authToken)
 case "apikey":
 req.Header.Set("X-API-Key", c.authToken)
 }

 // Send request
 client := &http.Client{}
 // TODO: Add SSL verification config
 resp, err := client.Do(req)
 if err != nil {
 return fmt.Errorf("failed to send webhook: %w", err)
 }
 defer resp.Body.Close()

 // Check response
 if resp.StatusCode < 200 || resp.StatusCode >= 300 {
 return fmt.Errorf("webhook returned status %d",
 resp.StatusCode)
 }

 return nil
}

func (c *WebhookChannel) GetName() string {
 return c.name
}

func (c *WebhookChannel) IsEnabled() bool {
 return c.enabled
}

func (c *WebhookChannel) GetConfig() map[string]interface{} {
 return map[string]interface{}{
 "url": c.url,
 "method": c.method,
 "headers": c.headers,
 "auth_type": c.authType,
 "verify_ssl": c.verifySSL,
 }
}

```

### Configuration in config.yaml:

```

notifications:
 channels:

```

```

custom_webhook:
 enabled: true
 url: "https://api.example.com/webhooks/helix"
 method: "POST"
 headers:
 X-Custom-Header: "value"
 auth_type: "bearer"
 auth_token: "${CUSTOM_WEBHOOK_TOKEN}"
 payload_template: |
 {
 "event": "helix_notification",
 "title": "{{.Title}}",
 "message": "{{.Message}}",
 "severity": "{{.Type}}",
 "timestamp": "{{.CreatedAt}}"
 }

```

**Setup Guide:** Create docs/integrations/GENERIC\_WEBHOOKS\_SETUP.md

**Acceptance Criteria:** - Generic webhook channel implemented - Custom templates supported - Multiple auth methods - Custom headers - Tests pass - Setup guide created

---

## Day 4-5: Microsoft Teams

### Task 4.2: Implement Microsoft Teams (Workflows API)

**File:** helix\_code/internal/notification/teams.go

```
package notification
```

```

import (
 "bytes"
 "context"
 "encoding/json"
 "fmt"
 "net/http"
)

// TeamsChannel implements Microsoft Teams notifications via
// Workflows
type TeamsChannel struct {
 name string
 enabled bool

```



```

 workflowURL string
 }

 // NewTeamsChannel creates a Teams channel
 func NewTeamsChannel(workflowURL string) *TeamsChannel {
 return &TeamsChannel{
 name: "teams",
 enabled: workflowURL != "",
 workflowURL: workflowURL,
 }
 }

 // Send sends notification to Microsoft Teams
 func (c *TeamsChannel) Send(ctx context.Context, notification
 *Notification) error {
 if !c.enabled {
 return fmt.Errorf("teams channel disabled")
 }

 // Build Adaptive Card
 card := c.buildAdaptiveCard(notification)

 payload := map[string]interface{}{
 "type": "message",
 "attachments": []map[string]interface{}{
 {
 "contentType": "application/
vnd.microsoft.card.adaptive",
 "content": card,
 },
 },
 }

 jsonData, err := json.Marshal(payload)
 if err != nil {
 return fmt.Errorf("failed to marshal teams payload: %w",
 err)
 }

 resp, err := http.Post(c.workflowURL, "application/json",
 bytes.NewReader(jsonData))
 if err != nil {
 return fmt.Errorf("failed to send to teams: %w", err)
 }
 defer resp.Body.Close()
 }

```

```

 if resp.StatusCode != http.StatusOK && resp.StatusCode !=
 http.StatusAccepted {
 return fmt.Errorf("teams returned status %d",
 resp.StatusCode)
 }

 return nil
}

// buildAdaptiveCard creates an Adaptive Card for the
// notification
func (c *TeamsChannel) buildAdaptiveCard(notification
 *Notification) map[string]interface{} {
 color := c.getColorForType(notification.Type)

 card := map[string]interface{}{
 "type": "AdaptiveCard",
 "version": "1.4",
 "body": []map[string]interface{}{
 {
 "type": "Container",
 "style": c.getStyleForType(notification.Type),
 "items": []map[string]interface{}{
 {
 "type": "TextBlock",
 "text": notification.Title,
 "weight": "Bolder",
 "size": "Large",
 "wrap": true,
 },
 {
 "type": "TextBlock",
 "text": notification.Message,
 "wrap": true,
 },
 },
 },
 },
 "msteams": map[string]interface{}{
 "width": "Full",
 },
 }

 // Add metadata if present

```

```

 if len(notification.Metadata) > 0 {
 facts := make([]map[string]interface{}, 0)
 for key, value := range notification.Metadata {
 facts = append(facts, map[string]interface{}{
 "title": key,
 "value": fmt.Sprintf("%v", value),
 })
 }

 card["body"] = append(card["body"].
 ([]map[string]interface{}), map[string]interface{}{
 "type": "FactSet",
 "facts": facts,
 })
 }

 return card
}

// getColorForType returns the theme color for notification type
func (c *TeamsChannel) getColorForType(notifType
 NotificationType) string {
 colors := map[NotificationType]string{
 NotificationTypeSuccess: "good",
 NotificationTypeWarning: "warning",
 NotificationTypeError: "attention",
 NotificationTypeAlert: "attention",
 NotificationTypeInfo: "default",
 }

 if color, exists := colors[notifType]; exists {
 return color
 }
 return "default"
}

// getStyleForType returns the container style
func (c *TeamsChannel) getStyleForType(notifType
 NotificationType) string {
 styles := map[NotificationType]string{
 NotificationTypeSuccess: "good",
 NotificationTypeWarning: "warning",
 NotificationTypeError: "attention",
 NotificationTypeAlert: "attention",
 }

```

```

 NotificationTypeInfo: "default",
 }

 if style, exists := styles[notifType]; exists {
 return style
 }
 return "default"
}

func (c *TeamsChannel) GetName() string {
 return c.name
}

func (c *TeamsChannel) IsEnabled() bool {
 return c.enabled
}

func (c *TeamsChannel) GetConfig() map[string]interface{} {
 return map[string]interface{}{
 "workflow_url": c.maskURL(c.workflowURL),
 }
}

func (c *TeamsChannel) maskURL(url string) string {
 if len(url) <= 20 {
 return "*****"
 }
 return url[:20] + "*****"
}

```

**Setup Guide:** Create docs/integrations/TEAMS\_SETUP.md

### IMPORTANT NOTE in setup guide:

# Microsoft Teams Integration Setup

## IMPORTANT: Webhook Deprecation

**\*\*As of December 2025, Microsoft is deprecating the old Office 365 Connectors.\*\***

You **MUST** use the new Workflows API. Old webhook URLs will stop working!

## Setup Steps

### ### 1. Create Power Automate Workflow

1. Go to <https://make.powerautomate.com/>
2. Create new flow: "When a HTTP request is received"
3. Configure to post to Teams channel
4. Copy the HTTP POST URL

### ### 2. Configure HelixCode

[Continue with setup...]

**Acceptance Criteria:** - Teams integration using Workflows API - Adaptive Cards for rich formatting - Color-coded by notification type - Metadata displayed - Setup guide with deprecation warning - Tests pass

---

## Phase 5: Advanced Integrations

**Duration:** Week 8-9 (10 days) **Priority:** LOW-MEDIUM **Dependencies:** Phase 1-4

**Goal:** Add PagerDuty, Jira, GitHub Issues

### Overview

Each integration follows the same pattern: 1. Implement NotificationChannel interface 2. Create comprehensive tests 3. Create setup guide 4. Update config.yaml schema 5. Update website

### Day 1-3: PagerDuty Integration

**File:** helix\_code/internal/notification/pagerduty.go

```
package notification
```

```
import (
 "bytes"
 "context"
 "encoding/json"
 "fmt"
 "net/http"
)
```

```
// PagerDutyChannel implements PagerDuty Events API v2
```

```
type PagerDutyChannel struct {
 name string
 enabled bool
 routingKey string
```

```

 apiURL string
}

func NewPagerDutyChannel(routingKey string) *PagerDutyChannel {
 return &PagerDutyChannel{
 name: "pagerduty",
 enabled: routingKey != "",
 routingKey: routingKey,
 apiURL: "https://events.pagerduty.com/v2/enqueue",
 }
}

func (c *PagerDutyChannel) Send(ctx context.Context,
 notification *Notification) error {
 if !c.enabled {
 return fmt.Errorf("pagerduty channel disabled")
 }

 // Build PagerDuty event
 event := map[string]interface{}{
 "routing_key": c.routingKey,
 "event_action": c.getEventAction(notification),
 "payload": map[string]interface{}{
 "summary": notification.Title,
 "source": "HelixCode",
 "severity": c.getSeverity(notification.Type),
 "custom_details": notification.Metadata,
 },
 }

 jsonData, err := json.Marshal(event)
 if err != nil {
 return fmt.Errorf("failed to marshal pagerduty event:
 %w", err)
 }

 resp, err := http.Post(c.apiURL, "application/json",
 bytes.NewReader(jsonData))
 if err != nil {
 return fmt.Errorf("failed to send to pagerduty: %w", err)
 }
 defer resp.Body.Close()

 if resp.StatusCode != http.StatusAccepted {

```

```

 return fmt.Errorf("pagerduty returned status %d",
 resp.StatusCode)
 }

 return nil
}

func (c *PagerDutyChannel) getEventAction(notification
 *Notification) string {
 // For errors/alerts, trigger incident
 if notification.Type == NotificationTypeError ||
 notification.Type == NotificationTypeAlert {
 return "trigger"
 }
 // For success, could resolve if we have incident key
 return "trigger"
}

func (c *PagerDutyChannel) getSeverity(notifType
 NotificationType) string {
 severities := map[NotificationType]string{
 NotificationTypeAlert: "critical",
 NotificationTypeError: "error",
 NotificationTypeWarning: "warning",
 NotificationTypeInfo: "info",
 NotificationTypeSuccess: "info",
 }
 return severities[notifType]
}

// Implement interface...

```

**Setup Guide:** docs/integrations/PAGERDUTY\_SETUP.md

**Acceptance Criteria:** - PagerDuty Events API v2 implemented - Incident creation works - Severity mapping correct - Deduplication key support - Setup guide complete - Tests pass

---

## Day 4-6: Jira Integration

**File:** helix\_code/internal/notification/jira.go

Implementation includes: - Create issue on error/failure - Update issue on status change - Add comments to existing issues - Custom field mapping

**Setup Guide:** docs/integrations/JIRA\_SETUP.md

---

## Day 7-9: GitHub Issues Integration

**File:** helix\_code/internal/notification/github.go

Implementation includes: - Create issue on failure - Add labels based on notification type - Add comments with execution details - Link to task/workflow

**Setup Guide:** docs/integrations/GITHUB\_SETUP.md

---

## Day 10: Documentation & Testing

- Complete all setup guides
  - Integration tests for all new channels
  - Update main documentation
  - Update website
- 

## Phase 6: Documentation & Website Polish

**Duration:** Week 10 (5 days) **Priority:** MEDIUM **Dependencies:** Phase 1-5 **Goal:** Polished documentation and website

### Tasks:

1. **API Documentation** (Day 1-2)
  - OpenAPI/Swagger spec
  - REST API endpoints
  - Webhook formats
  - Authentication
2. **Configuration Reference** (Day 2-3)
  - Complete config.yaml reference
  - Environment variables guide
  - Rule syntax documentation
  - Template guide
3. **Update Website** (Day 3-4)
  - Integration showcase
  - Interactive setup wizard (optional)
  - Code examples
  - FAQ section
4. **Video Tutorials** (Day 4-5, optional)
  - Slack setup walkthrough



- Telegram setup walkthrough
  - Event-driven notifications demo
- 

## Phase 7: Performance & Scale Testing

**Duration:** Week 11 (5 days) **Priority:** LOW **Dependencies:** All previous phases

**Goal:** Ensure system can handle scale

### Day 1-2: Load Testing

**File:** helix\_code/test/performance/notification\_load\_test.go

```
//go:build performance
```

```
package performance
```

```
import (
 "context"
 "testing"
 "time"

 "dev.helix.code/internal/notification"
)
```

```
func TestNotificationThroughput(t *testing.T) {
 // Test: Can system handle 1000 notifications/minute?

 engine := notification.NewNotificationEngine()
 // ... setup

 notifications := 1000
 start := time.Now()

 for i := 0; i < notifications; i++ {
 notif := ¬ification.Notification{
 Title: fmt.Sprintf("Test %d", i),
 Message: "Load test",
 Type: notification.NotificationTypeInfo,
 }
 engine.SendNotification(context.Background(), notif)
 }

 duration := time.Since(start)
```

```
throughput := float64(notifications) / duration.Seconds()

t.Logf("Sent %d notifications in %v (%.2f/sec)",
 notifications, duration, throughput)

// Assert minimum throughput
assert.True(t, throughput > 16.67, "Should handle at least
 1000/min (16.67/sec)")
}
```

## Day 3-4: Optimization

- Connection pooling
- Batch sending (where applicable)
- Caching
- Database query optimization

## Day 5: Benchmark & Report

- Run benchmarks
- Create performance report
- Document optimization
- Set baseline metrics

---

# Quick Reference: Daily Tasks

## Every Day:

### 1. Morning:

- Review previous day's work
- Run all tests: `go test ./... -v`
- Check CI/CD status

### 2. During Development:

- Write tests FIRST (TDD)
- Run tests frequently
- Update documentation as you code
- Commit small, logical changes

### 3. End of Day:

- Run full test suite
- Update task status
- Document any blockers
- Push code

## Weekly:

- Review overall progress vs roadmap
  - Update stakeholders
  - Refactor as needed
  - Update documentation
- 

## Success Criteria

### Phase 1:

- All tests running in CI/CD
- Mock servers working
- Integration tests passing
- 100% test coverage maintained

### Phase 2:

- Events trigger notifications automatically
- No manual notification calls needed
- All components emit events
- E2E tests prove the flow

### Phase 3:

- 99.9% notification delivery rate
- No lost notifications
- Graceful degradation under load
- Observability dashboards

### Phase 4-5:

- Each integration fully functional
- Setup guide for each
- Tests passing
- Website updated

### Phase 6-7:

- Documentation complete
  - Performance benchmarks met
  - System ready for production
-

# Troubleshooting Guide

## Tests Failing:

1. Check error messages carefully
2. Run single test: `go test -run TestName`
3. Check test dependencies (Redis, etc.)
4. Verify mock servers running

## Integration Issues:

1. Check API keys/tokens
2. Verify network connectivity
3. Check rate limits
4. Review logs

## Performance Issues:

1. Profile with pprof
  2. Check Redis performance
  3. Monitor goroutine count
  4. Check for memory leaks
- 

## Next Steps Tomorrow

### Start with Phase 1, Day 1:

1. Create `helix_code/internal/notification/testutil/` directory
2. Implement `mock_servers.go`
3. Write tests for mock servers
4. Verify tests pass
5. Update this document with progress

### Command to run:

```
cd /home/milosvasic/Projects/HelixDevelopment/helix_code/
 HelixCode
mkdir -p internal/notification/testutil
Start implementing Task 1.1
```

---

**Document Version:** 1.0 **Last Updated:** 2025-11-04 **Total Estimated Time:** 11 weeks **Current Phase:** Phase 0 Complete, Ready for Phase 1